

Звіт з навантажувального тестування REST API

1. Мета тестування

Перевірити стабільність та продуктивність REST API-додатку при різних сценаріях навантаження, оцінити час відповіді, пропускну здатність та використання ресурсів системи.

2. Інструменти та середовище

Компонент	Опис
JMeter	Apache JMeter 5.6.3
Plugins	Custom Thread Groups, PerfMon Metrics Collector
ServerAgent	PerfMon ServerAgent 2.2.3 (порт 4444)
Мова бекенду	Node.js (локальний сервер node ./server.js)
ОС / Пристрій	macOS Monterey, MacBook Pro (2015)
Тестоване API	CRUD-операції /characters (GET, POST, PUT, DELETE)

3. Структура тест-плану

Test Plan: REST CRUD – 100 Requests. Використано Stepping та Ultimate Thread Group, Duration Assertion, JSON Extractor, Aggregate Report, PerfMon Metrics Collector.

4. Сценарії навантаження

Stepping TG: 5 користувачів, 20 сек, 4 лупи (~100 запитів). Ultimate TG: 5 користувачів, 30 сек Hold, 5 сек старт/стоп.

5. Результати тестування

Aggregate Report: середній час відповіді 1-2 мс, Throughput ~4000 req/s, Error % = 0%.

6. Моніторинг системних ресурсів (PerfMon)

CPU 30-40%, Memory ~3.6 млн (≈3 GB), Network стабільний, Disk I/O мінімальний. Система працює стабільно без піків навантаження.

9. Аналіз результатів

Система стабільна при навантаженні: середній час відповіді 1-2 мс, Throughput ~4000 req/s, Error 0%. CPU та Memory показали сталу роботу без перевищень. API обробляє запити ефективно.

10. Потенційні вузькі місця

Категорія	Можлива проблема	Ознака у звіті
Серверна частина	Однопоточковість при великому навантаженні	CPU > 70%
База даних	Повільні операції запису/читання	Піки часу відповіді POST/PUT
JMeter	Надлишкові Listener-и або логування	Пам'ять JVM > 3 GB

Мережа	Латентність між клієнтом і сервером	Зниження Throughput
--------	-------------------------------------	---------------------

11. Рекомендації з оптимізації

Оптимізація JMeter: 1. Залишати тільки Aggregate Report і PerfMon. 2. Використовувати non-GUI режим: `jmeter -n -t test.jmx -l results.jtl -e -o /reports/html`. 3. Обмежити логування через `jmeter.properties`. 4. Збільшити `heap` до `-Xms1g -Xmx4g`. Оптимізація API: 1. Використати PM2/Cluster Mode. 2. Кешувати GET-запити. 3. Оптимізувати базу даних (індекси, асинхронність). 4. Вимкнути докладне логування у production.

13. Висновок

REST API стабільно обробляє до 4000 запитів/сек без помилок. Середній час відповіді <2 мс. CPU та пам'ять у межах норми. Конфігурація готова до масштабування, стрес-тестів та продовження оптимізації.